Layer-wise Incremental Training of Variational Quantum Circuits to Mitigate Barren Plateaus

Complete Viva / Project-Review Study Guide

Covers everything from basic quantum-computing concepts through the actual implemented code, the real executed results, and a full anticipated-questions bank -- written for a 3-person team to study from and present without notes.

Dataset: UCI Statlog German Credit Data (id 144)

Framework: PennyLane (Python 3.13, CPU simulators only)

Repository: github.com/Tharungk2005/Barren_plateau-in-VQC

Table of Contents

1. Quantum Computing Basics You Need for This Project
2. Variational Quantum Circuits and Quantum Machine Learning
3. The Barren Plateau Problem
4. Project Overview: Title, Problem, Research Questions
5. The Dataset
6. Data Preprocessing Pipeline
7. Circuit Architecture (Mathematical Detail)
8. Loss, Optimizer, and Gradient Computation
9. Training Protocol A: Direct Training
10. Training Protocol B: Layer-wise Incremental Training
11. Why Layer-wise Training Is Hypothesised to Help
12. Experimental Design
13. Evaluation Metrics -- What and Why
14. Classical Baselines
15. Results (Actual Numbers, Quick Mode)
16. Analysis and Honest Interpretation
17. Limitations (List These Proactively, Don't Wait to Be Asked)
18. The Repository, File by File
19. Every Table Explained, Column by Column
20. Every Figure Explained, Type by Type
21. Live Demonstration Script
22. Suggested Division Among 3 Team Members (for the live talk only -- everyone must still know all of it for Q&A;)
23. Anticipated Viva Questions and Model Answers
24. Glossary

1. Quantum Computing Basics You Need for This Project

This section covers only the quantum-computing background that this project actually uses. It is deliberately narrow -- it does not try to teach general quantum computing.

1.1 Qubit and quantum state

A classical bit is 0 or 1. A qubit's state is a vector in a 2-dimensional complex vector space, written $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$, where α and β are complex numbers satisfying $|\alpha|^2 + |\beta|^2 = 1$. $|0\rangle$ and $|1\rangle$ are the two basis states (analogous to 0 and 1). A system of n qubits lives in a 2^n-dimensional space -- this exponential growth is why even 4-8 qubits can represent a rich function, and also why simulating many more qubits on a classical computer becomes expensive.

1.2 Gates used in this project

- **RY(θ)** -- rotates the qubit's state around the Y-axis of the Bloch sphere by angle θ. It is a single-qubit, single-parameter gate. In this project it is used twice: once to encode a classical input feature into the qubit (the 'embedding' / 'encoding' gate), and once per layer as a trainable rotation.
- **RZ(φ)** -- rotates the qubit's state around the Z-axis by angle φ. Used as the second trainable rotation per qubit per layer. RY and RZ together let a single qubit's state be rotated to any point reachable by two independent real parameters (a full single-qubit rotation needs 3 real parameters in general, e.g. RZ-RY-RZ; this project deliberately uses only 2 per qubit per layer, which keeps the parameter count exactly 2n per layer and is a design choice, not a limitation that breaks the architecture).
- **CZ (Controlled-Z)** -- a 2-qubit gate with no trainable parameter. It applies a phase flip (multiplies the amplitude by -1) to the $|11\rangle$ component of the two qubits it acts on, and leaves $|00\rangle$, $|01\rangle$, $|10\rangle$ unchanged. This is what **entangles** the qubits: after a CZ gate, the two qubits' states can no longer be written as a simple product of two independent single-qubit states in general. Without any entangling gate, the whole n-qubit circuit would just be n independent single-qubit rotations -- mathematically equivalent to n completely separate 1-feature models, unable to capture any interaction between input features. CZ is what makes this a genuine multi-qubit circuit.

1.3 Measurement and expectation value

You cannot read out α and β directly from a real qubit. You can only measure it, which probabilistically collapses it to $|0\rangle$ or $|1\rangle$. The Pauli-Z operator has eigenvalues +1 (for $|0\rangle$) and -1 (for $|1\rangle$). Its **expectation value**, written $\langle\psi|Z|\psi\rangle$, is the probability-weighted average outcome over many repeated measurements ("shots"): $\langle Z\rangle = P(0) \cdot (+1) + P(1) \cdot (-1)$. This value is always a real number in the range [-1, +1]. On a real quantum computer you estimate it by running the circuit many times and averaging the measured +1/-1 outcomes -- that is exactly what the project's "finite shots" experiment simulates (Section 12).

1.4 Simulators used: exact vs. finite-shot vs. noisy

- **default.qubit, analytic mode** -- PennyLane computes the exact statevector and the exact expectation value with no sampling randomness, as if you had infinite shots. This is what "Direct Training" and "Layer-wise Training" use for the main (noiseless) experiment.

- **default.qubit, finite shots** -- the same simulator but it simulates a real device's measurement statistics: it draws a finite number of samples (e.g. 100 or 1000) from the exact probability distribution and reports the sample average, which has statistical noise around the true expectation value.
- **default.mixed** -- a density-matrix simulator, needed when the circuit includes noise channels (Section 12.3). It tracks a mixed quantum state (a probabilistic combination of pure states), which is required to represent the effect of noise.

2. Variational Quantum Circuits and Quantum Machine Learning

A **Variational Quantum Circuit** (VQC), also called a Parameterized Quantum Circuit (PQC) or (loosely) a Quantum Neural Network, is a quantum circuit that contains tunable, classical real-number parameters θ (rotation angles). You run the circuit, measure an observable, get a number out, compare it to a target using a classical loss function, and use a classical optimizer to update θ -- exactly the same training loop structure as a classical neural network, except the function being optimized is evaluated by a quantum circuit instead of classical matrix multiplications.

2.1 The hybrid quantum-classical loop

1. A classical input x (here, a 4- or 6-dimensional vector of PCA features scaled to [0, π]) is encoded into the quantum circuit using fixed (non-trainable) gates -- here, RY(x_i) on each qubit i.
2. Trainable gates (RY(θ), RZ(φ), and the fixed CZ entanglers) are applied -- this is the 'ansatz' or 'variational form'.
3. One qubit is measured; its Pauli-Z expectation value f(x, θ) ∈ [-1, +1] is the circuit's output.
4. A classical loss function compares f(x, θ) to the true label y and produces a scalar cost.
5. A classical optimizer (here, Adam) computes the gradient of the cost with respect to θ and updates θ to reduce the cost.
6. Steps 1-5 repeat for many batches of training data until a stopping criterion (a fixed update budget, in this project) is reached.

2.2 Why this is called 'variational'

The term comes from the variational principle in physics: you propose a parameterized family of quantum states/circuits and search over the parameters to optimize some objective, rather than solving the problem exactly. The circuit's expressive power -- which functions it can represent -- depends on its depth (number of layers), width (number of qubits), and which gates it uses.

2.3 Why use a quantum circuit for this at all

This project does not claim a practical quantum advantage on this task -- it explicitly is run entirely on classical simulators (Section 18, Limitations). The scientific purpose is narrower and specific: to study a known **training pathology** of VQCs (the barren plateau, Section 3) and test whether a specific training strategy (layer-wise growth) mitigates it, on a real (not synthetic) classification task, as a controlled experiment.

3. The Barren Plateau Problem

3.1 Definition

A **barren plateau** is a region of the parameter landscape where the gradient of the cost function with respect to (almost) every parameter is extremely small -- small enough that it is indistinguishable from the noise introduced by finite sampling (shots) and floating-point precision. When an optimizer's gradient signal is swamped by this noise floor, it cannot tell which direction improves the cost, so training stalls, often right from a random initialization.

3.2 Why it happens (McClean et al., 2018)

McClean et al. proved that for circuits whose structure makes them behave like Haar-random unitaries over the space they explore (technically: circuits forming an approximate unitary 2-design), the **variance** of the gradient of a typical parameter shrinks **exponentially** with the number of qubits n. Since the gradient's mean is zero by symmetry in that setting, a shrinking variance means the gradient is overwhelmingly likely to be extremely close to zero for almost every random parameter setting, for almost every parameter -- hence 'barren plateau'. The effect gets worse as the circuit gets wider (more qubits) and, separately, as it gets deeper (more layers), because deeper random circuits approach 2-design behaviour faster.

3.3 Other contributing causes identified in later work

- **Cost-function locality** (Cerezo et al., 2021): if the measured observable acts non-trivially on **all** qubits ('global' cost), barren plateaus appear even in shallow circuits. If it acts on only a **few** qubits ('local' cost -- e.g. this project's single-qubit PauliZ(0) readout), the plateau onset is delayed to larger depths. This is exactly why this project's circuit measures only wire 0, not a multi-qubit observable.
- **Hardware/simulated noise** (Wang et al., 2021): under local depolarizing noise, gradients vanish exponentially in qubit count once circuit depth scales with qubit count, because noise accumulates with every gate layer. This motivates this project's depolarizing-noise sub-experiment (Section 12.3).
- **High expressibility / entanglement** (surveyed in Larocca et al., 2025): ansatze that can reach a very large, 'generic' portion of Hilbert space tend to suffer worse plateaus -- there is a trade-off between how expressive an ansatz is and how trainable it is.

3.4 Why this matters practically

If gradients are unresolvably small, no classical optimizer -- however well-tuned -- can make progress, regardless of how good the true optimum is. This is a fundamental trainability obstacle for scaling VQCs to more qubits/layers, separate from whether a good solution exists in principle.

3.5 This project's own direct evidence (the gradient-variance diagnostic)

Independent of the credit-risk dataset, this project runs a dedicated diagnostic (Section 12.4, Section 15.7): for each of several (qubit count, depth) combinations, it draws many random parameter settings (uniform [0, 2π), the same distribution Direct Training uses to initialize), computes the exact gradient of one representative parameter via autodiff, and measures the **variance** of that gradient across the random draws. The measured result: variance drops from about 0.49-0.52 at depth 1 to 0.032 (4 qubits), 0.0081 (6 qubits), and 0.0019 (8 qubits) at depth 24 -- both depth and qubit-count make it worse, exactly matching the theory. This is the project's own concrete, reproducible evidence that its architecture really does exhibit a barren-plateau effect, not just an assumption borrowed from the literature.

4. Project Overview: Title, Problem, Research Questions

4.1 Title

Layer-wise Incremental Training of Variational Quantum Circuits to Mitigate Barren Plateaus

4.2 Problem statement

Barren plateaus can make VQC training landscapes effectively untrainable as circuits scale. This project investigates whether **gradually increasing circuit depth through layer-wise training** improves trainability compared with training a complete, deep circuit from the start -- on a real classification task, not a toy problem.

4.3 Primary research question

On a real-world credit-risk classification task, does layer-wise incremental training of a VQC improve trainability, convergence, predictive performance, or robustness to sampling and simulated quantum noise, compared with direct training?

4.4 Secondary research questions

1. How does circuit depth affect gradient magnitudes and training behaviour?
2. Does layer-wise training perform differently for shallow vs. deep circuits?
3. How sensitive are both methods to finite measurement shots?
4. How sensitive are both methods to simulated depolarizing (hardware-like) noise?
5. What are the trade-offs in accuracy, convergence speed, circuit evaluations, and computation time?
6. Do any observed improvements remain consistent across random seeds?

4.5 The one scientific rule that shaped every design decision

The project was required to investigate honestly, not to prove layer-wise training wins. Every experiment was predeclared (grid sizes, seeds, statistical test, convergence target, success-rate definition) before looking at results, and the report states where Direct Training actually won, where results were statistically inconclusive, and where the dataset itself (not the quantum model) is the limiting factor. This is not just an ethical nicety -- it is what the whole report and this Q&A; guide are built around, and an examiner can ask you to justify any specific number precisely because nothing was cherry-picked.

5. The Dataset

5.1 What it is

UCI Machine Learning Repository, **Statlog (German Credit Data)**, dataset id 144 (archive.ics.uci.edu/dataset/144), DOI 10.24432/C5NC77. 1000 loan applicants, 20 raw attributes (13 categorical, 7 numerical), binary target: class 1 = good credit risk (700 instances), class 2 = bad credit risk (300 instances). No missing values (verified programmatically).

5.2 Why this dataset is appropriate here

- It is real, not synthetic -- the project's rules explicitly forbid generating fake predictive data.
- It is small enough (1000 rows) to train many configurations quickly on a CPU simulator.
- It genuinely has more raw features (20, expanding to 61 after one-hot encoding) than any qubit count tested (4, 6, 8), which forces a real dimensionality-reduction step -- this is representative of the near-term reality that VQCs have very few qubits relative to typical classical feature counts.
- Its documented, official cost matrix (misclassifying a bad-risk applicant as good costs 5x more than the reverse mistake) supports a genuine cost-sensitive metric, not an invented one.

5.3 Honest limitations of this dataset (say this unprompted, it shows rigor)

- 1000 rows is small by modern ML standards.
- 700/300 class imbalance means a trivial always-predict-good-credit model already scores 70% raw accuracy -- which is exactly the trap the project's own results fall into for Direct Training (Section 16.2).
- Many features are coarse ordinal/categorical buckets, not continuous measurements.

5.4 Label convention used throughout the entire codebase

UCI class	Meaning	Project-internal label
1	Good credit	-1
2	Bad credit	+1 (the positive class for precision/recall/F1/cost)

A prediction is "bad credit" if the circuit's output $f(x,\theta) > 0$, otherwise "good credit". This threshold-at-zero rule is used identically everywhere results are reported.

5.5 Official UCI cost matrix (used verbatim, not invented)

Actual \ Predicted	Good	Bad
Good	0	1
Bad	5	0

Rows = actual class, columns = predicted class. Missing a bad-risk applicant (predicting them "good") costs 5x more than wrongly flagging a good applicant as bad -- reflecting that losing the loan principal is worse than losing a potential customer.

6. Data Preprocessing Pipeline

6.1 Split

A single, fixed, stratified 70% / 15% / 15% train / validation / test split (random seed fixed in config.json), used identically by every qubit count, every protocol, and the classical baselines. Stratified means the 70/300 class ratio is preserved in every split. Result sizes: 700 / 150 / 150.

6.2 Why 'leakage-safe' matters and how it is enforced

Data leakage means information from validation or test data influences how the training pipeline is fit, which makes reported performance optimistically biased and not representative of truly unseen data. Every fitted transform in this pipeline -- one-hot encoder, standard scaler, PCA, and the angle min-max scaler -- is fit **exclusively on the training split**, then only **applied** (never refit) to validation and test data. This is directly unit-tested (test_pca_fit_on_train_only_no_leakage) and verified with disjoint-index assertions.

6.3 Step-by-step pipeline

1. **Categorical encoding**: 13 categorical attributes (e.g. 'purpose of loan', 'housing') are one-hot encoded (each category becomes its own 0/1 column); unseen categories at validation/test time are safely handled (encoded as all-zeros) rather than crashing.
2. **Numerical scaling**: 7 numerical attributes (e.g. age, credit amount, duration) are standardized (z-score: subtract the training mean, divide by the training standard deviation).
3. **Concatenation**: the one-hot columns and scaled numerical columns are concatenated into one 61-dimensional vector per applicant.
4. **PCA (Principal Component Analysis)**: the 61-dimensional vector is linearly projected down to n_qubits dimensions (4, 6, or 8) by keeping the n_qubits directions of largest variance in the training data. This is **necessary** because the circuit has exactly one encoding gate per qubit -- the number of input features must equal the number of qubits.

5 **Angle scaling**: each of the n_qubits PCA components is independently min-max scaled, using training-split minimum/maximum only, to the range [0, π]. Validation/test values that fall outside the training-observed range are clipped into [0, π] (documented, not silently wrong). This range is chosen because it is exactly the range needed to make RY(angle) sweep a qubit from |0⟩ (angle=0) to |1⟩ (angle=π) -- using the full useful range of the rotation without wrapping past 2π redundantly.

6.4 Information loss is real and quantified -- say this proactively

PCA to 4 qubits retains only **41.4%** of the processed feature variance; PCA to 6 qubits retains **53.8%**. This single number is the most honest explanation for most of the gap between the VQC's performance and the full-feature classical baseline (Section 16.6) -- more than half the information in the processed features is discarded before the quantum circuit ever sees the data, purely because there are only 4-8 qubits to encode into.

7. Circuit Architecture (Mathematical Detail)

7.1 The full circuit, layer by layer

For n qubits and input angles $x \in [0, \pi]^n$:

```
|ψ(x,θ)⟩ = [ Prod_{l=L}^{1} U_ent · U_rot(θ_l) ] · U_enc(x) |0⟩^n
```

- **Encoding, U_enc(x)**: apply RY(x_i) on qubit i, for every i = 0..n-1. This happens once, before any trainable layer.
- **Per-layer rotation, U_rot(θ_l)**: on every qubit i, apply trainable RY(θ_{l,i,1}) then trainable RZ(θ_{l,i,2}). That is 2 trainable angles per qubit per layer.
- **Per-layer entanglement, U_ent**: apply a fixed (non-trainable) CZ gate on every edge of a 'ring' connecting neighbouring qubits: (0,1), (1,2), ..., (n-2,n-1), (n-1,0). For exactly 2 qubits, the ring has only one distinct edge (0,1) -- the code explicitly avoids applying it twice, which the test suite checks directly (test_entangling_edges_two_qubits_no_duplicate).
- **Readout**: measure expval(PauliZ(wire 0)) only -- a single qubit, a 'local' observable (Section 3.3) -- giving the circuit's output f(x,θ) ∈ [-1, +1].

7.2 Trainable parameter count

```
p = 2 × n_qubits × n_layers
```

CZ gates contribute zero trainable parameters (they are fixed). This formula is directly unit-tested (test_param_count_formula) and matches every circuit diagram generated by the project: e.g. 4 qubits, 2 layers → p = 16; 6 qubits, 12 layers → p = 144.

7.3 The architectural caveat you must be able to explain precisely

When Layer-wise Training inserts a new layer, its new RY/RZ angles are initialized **near zero** (uniform in [-0.01, +0.01] radians), not at exactly zero, and not with the wide [0, 2π) range used for the very first layer. **A natural but WRONG assumption is that near-zero rotation angles make the new layer act as an identity operation (i.e. do nothing).** This is false here, because the layer's CZ entangling gates are applied **unconditionally** -- they do not depend on any trainable angle at all, so they are always 'on' regardless of how close the rotations are to zero. A CZ gate still changes the relative phase of the |11⟩ component even when the surrounding rotations are infinitesimally small. The project measured this directly: for a real 4-qubit, 12-layer run, inserting layer 2 (with its near-zero angles, before any training of that layer) shifted the validation MSE loss from 1.202706 to 1.202672 -- small, but non-zero and reproducibly measured, not simulated or assumed. This caveat is also why the project freezes previous layers with an explicit mask (not just 'hoping' the near-zero layer leaves them undisturbed in effect) -- freezing guarantees the old parameters literally cannot change, even though the circuit's output does shift slightly.

7.4 Why RY for encoding specifically

RY is a real-valued rotation (no complex phase introduced on its own), so a real classical number maps cleanly and monotonically onto a qubit's state along one great circle of the Bloch sphere. It is one of the most common, simplest 'angle encoding' choices in the QML literature for exactly this reason.

8. Loss, Optimizer, and Gradient Computation

8.1 Loss function: Mean Squared Error

$$MSE(\theta) = (1/|B|) \times \Sigma_{(x,y)\in B} (f(x,\theta) - y)^2$$

y is the project-internal label (-1 or +1, Section 5.4), f(x,θ) is the circuit's output (also in [-1,+1]), and B is a training mini-batch. MSE was chosen (rather than, say, a sigmoid + cross-entropy) specifically because the circuit's natural output range is already [-1,+1] -- matching it directly to a ±1 label avoids needing an extra classical post-processing layer on top of the quantum circuit.

8.2 Optimizer: Adam

Adam (Adaptive Moment Estimation) maintains a running exponential average of each parameter's past gradients (first moment, like momentum) and of their squared magnitudes (second moment), and uses those to give each parameter its own adaptive effective learning rate. Learning rate = 0.01 (configurable). This project implements Adam manually in plain NumPy (not via PennyLane's built-in optimizer) specifically so that, during Layer-wise Training, the optimizer state and update can be scoped to **only the newly added layer's parameters** while older layers are mathematically excluded from the gradient computation entirely (Section 10.3).

8.3 Two different ways to get the gradient -- and why both are needed

(a) Backpropagation / automatic differentiation ('backprop')

Used whenever the simulation is exact (noiseless, infinite 'shots'). PennyLane's autograd-based simulator can differentiate straight through the simulated circuit, the same way PyTorch/TensorFlow differentiate a classical neural network. This is fast and exact, but it is a **simulator-only** trick -- a real quantum computer cannot literally 'backpropagate through' a physical measurement.

(b) The parameter-shift rule

Used whenever shots are finite or noise is present -- i.e. whenever the computation must resemble what a real device could actually do. For a gate like RY(θ), the exact derivative of the circuit's expectation value with respect to θ can be computed as:

```
∂f/∂θ = [ f(θ+π/2) - f(θ-π/2) ] / 2
```

i.e. by running the **whole circuit twice** more, at θ shifted by +π/2 and -π/2, and taking a simple half-difference -- no backpropagation needed, just two extra full circuit executions per parameter. This is exact (not an approximation) and is directly implementable on real quantum hardware. Its cost is why this project's finite-shots and noise sub-experiments are so much slower than the main noiseless grid: the cost of one gradient estimate scales as roughly 2 × (number of trainable parameters) extra circuit evaluations per training sample, per update -- for a 48-parameter circuit that is about 97 circuit evaluations just for one sample's gradient, versus effectively 1 for backprop. This is exactly what was measured in the pilot (Section 12.5) and is why the noise experiment's depth-6 configurations were dropped from quick mode.

9. Training Protocol A: Direct Training

9.1 Procedure

1. Build the **complete** L-layer circuit up front (all L layers exist from update 1).
2. Initialize **every** trainable angle independently and uniformly at random in [0, 2π) -- a reproducible NumPy random generator seeded per-run.
3. Train **all** 2 × n_qubits × L parameters together, jointly, using Adam, for the full update budget (150 updates in quick mode).
4. At every recorded checkpoint: log training loss, validation loss, validation accuracy, validation balanced accuracy, raw and parameter-count-normalized gradient norm, cumulative (estimated) circuit evaluations, and cumulative wall-clock time.

9.2 What Direct Training is meant to represent

This is the 'naive' or 'textbook' way to train a VQC -- exactly analogous to randomly initializing an entire deep classical neural network and training every layer together from scratch. It is the baseline that layer-wise training is compared against. If a barren plateau exists at the chosen depth, Direct Training starts directly inside (or very near) it, because its random initialization spans the full, deep parameter space from the first update.

10. Training Protocol B: Layer-wise Incremental Training

10.1 Procedure

1. Start with a circuit of **exactly 1 layer**. Initialize that layer's angles uniformly in [0, 2π) -- the same wide range Direct Training uses, since there is no previous solution to disturb yet.
2. Train **only that first layer** for budget/L updates (budget split evenly across L stages by default -- a documented, configurable rule).
3. **Freeze** that trained layer: its parameters are mathematically excluded from the next gradient computation (see 10.3) -- they genuinely cannot move anymore, not merely 'discouraged' from moving via a small learning rate.
4. **Insert a new layer**, its angles initialized **near zero** (uniform in [-0.01, +0.01] radians).
5. Train **only the new layer's** parameters for budget/L more updates, with every previous layer frozen.
6. Repeat insert-and-train-only-the-newest-layer until the target depth L is reached.
7. At every layer insertion: record the validation loss immediately before training that stage starts (with the new near-zero layer already active -- Section 7.3) and immediately after the stage finishes, so the loss perturbation caused by insertion, and the improvement from that stage's training, are both measured and saved, never assumed.

10.2 Fair-budget comparison

Both protocols receive the **same total number of optimizer updates** (150 in quick mode) for a given (qubit count, target depth) cell -- this is the predeclared, documented 'fair budget' rule (config.json, training.layerwise_budget_allocation = 'equal_per_stage'). Because the two protocols train different numbers of active parameters per update on average, the project also separately tracks and reports wall-clock time and (estimated) circuit evaluations, so it never silently implies equal compute just because the update **count** matches (Section 15.9).

10.3 How 'freezing' is actually implemented (important technical point)

This is not implemented by setting a near-zero learning rate on old parameters. Instead, during each stage, the circuit function is called with the frozen layers passed in as a **plain, non-differentiable NumPy array** (constant data) and only the newest layer's parameters passed in as the **differentiable** array that the automatic-differentiation engine (qml.grad) is told to differentiate with respect to. The gradient is therefore mathematically computed only with respect to the new layer's angles -- the old angles are concatenated into the forward pass (so their effect on the circuit's output is still fully present) but cannot appear in the gradient at all, and so Adam's update step cannot touch them. The project verifies this with an exact bit-for-bit equality check after every single stage (frozen_params_unchanged, asserted True in every real run, and separately covered by an automated test, test_layerwise_freezes_previous_layers) -- 'frozen' is a guarantee, not an approximation.

11. Why Layer-wise Training Is Hypothesised to Help

The connection to barren plateaus (Section 3) is direct: gradient variance was shown (both in theory and in this project's own diagnostic, Section 3.5) to shrink with **both** circuit depth and qubit count. Layer-wise Training's first stage is always a 1-layer circuit -- the shallowest, best-conditioned point on that curve, regardless of the eventual target depth. Every subsequent stage adds only a **single new layer** on top of an already-reasonably-trained circuit, rather than ever asking the optimizer to find a good direction from a single random draw deep inside a high-dimensional, potentially-barren landscape. In short: Direct Training gambles everything on one random draw in the full L-layer space; Layer-wise Training never has to take that gamble, because it only ever searches one shallow layer's worth of new parameters at a time, starting from a position that is already known to work reasonably well for the previous (shallower) depth.

This project does not simply assume this works -- it is exactly the hypothesis the whole experiment was designed to test honestly, and Section 16 reports a genuinely mixed, nuanced result, not a clean confirmation.

12. Experimental Design

12.1 Central configuration file

Every experimental setting (qubit counts, depths, seeds, update budget, learning rate, batch size, noise probabilities, shot counts, statistical test choice, convergence target definition) lives in one file, config.json, not scattered through the code. This is what makes the whole experiment 'predeclared' in the scientific-honesty sense of Section 4.5.

12.2 Main grid (the primary noiseless comparison) -- QUICK MODE, actually executed

Setting	Value
Qubit counts	4, 6
Depths (layers)	2, 6, 12
Protocols	Direct, Layer-wise
Seeds	0, 1, 2 (3 seeds)
Total optimizer-update budget	150 (same for both protocols)
Batch size	32
Optimizer / learning rate	Adam / 0.01
Gradient method	backprop (exact, noiseless)
Total runs	2 qubit counts × 3 depths × 2 protocols × 3 seeds = 36 runs

Result: all 36 runs completed successfully, 0 failures.

12.3 Finite-shots and depolarizing-noise sub-experiments

Scoped to a smaller, predeclared subset (4 qubits; shots at depths 2 and 6; noise at depth 2 only) for computational feasibility -- using parameter-shift gradients (Section 8.3b), which are much more expensive than backprop. Shots tested: exact/analytic, 1000 shots, 100 shots. Depolarizing noise probabilities tested: 0, 0.001, 0.01, applied as a DepolarizingChannel after every gate on every qubit, simulated via PennyLane's default.mixed density-matrix device.

12.4 Gradient-variance (barren plateau) diagnostic

A dataset-independent, circuit-only experiment: 4, 6, and 8 qubits × depths 1, 2, 4, 8, 12, 16, 20, 24, with 60 random circuit samples per (qubits, depth) cell in quick mode (200 in the not-yet-executed full mode). Reports variance, mean, and a 95% bootstrap confidence interval of a representative gradient component, plus the mean gradient-vector norm.

12.5 Pilot run: measuring feasibility before committing to a grid

Before the real grid was run, a pilot measured actual milliseconds-per-update at representative configurations (backprop, noiseless):

Qubits	Depth	Direct (ms/update)	Layer-wise (ms/update)
4	2	8.8	7.5
4	12	45.1	29.3
6	12	81.1	53.5
8	24	369.5	366.8

Extrapolated, the **full** grid (3 qubit counts × 4 depths up to 24 × 2 protocols × 10 seeds, 600-update budget) was estimated at roughly 3.5-4 hours of noiseless CPU time alone, before the shots/noise sub-experiments -- too long for this delivery, so it was left fully implemented, configured, and resumable, but not exhaustively executed. This is stated plainly everywhere results are reported, including in this document.

A second, live measurement during the noise sub-experiment found a single depth-2 (16-parameter) parameter-shift + density-matrix run took 124 seconds; depth-6 has 3× the parameters plus deeper simulation, so depth 6 was dropped from the quick-mode noise subset and this exact deviation is documented in config.json and the report -- an example of adjusting scope for **measured** feasibility, which the project's own rules explicitly permit, rather than silently cutting corners.

12.6 Full end-to-end measured run time (quick mode, everything included)

A complete, clean quick-mode run -- dataset download, preprocessing, classical baselines, all 36 main-grid runs, the shots and noise sub-experiments, the gradient diagnostic, circuit diagrams, and every table/figure -- actually completed in **355 seconds** (just under 6 minutes), recorded in results/quick/run_summary.json.

13. Evaluation Metrics -- What and Why

13.1 Why not just 'accuracy'

With a 700/300 class split, a model that **always** predicts 'good credit' scores 70% raw accuracy while being completely useless at its actual job (identifying bad-risk applicants). This project treats raw accuracy as a secondary, potentially misleading number, and explicitly found this exact failure mode in its own Direct Training results (Section 16.2) -- this is a central talking point, not a footnote.

13.2 Predictive-performance metrics

- **Balanced accuracy** = average of (true positive rate, true negative rate) -- treats both classes equally regardless of their frequency, so the always-predict-majority trap scores only ~50% (chance level), not 70%.
- **Precision, Recall, F1** -- computed with 'bad credit' (+1) as the positive class. Precision = of everyone flagged bad, how many really were; Recall = of everyone really bad, how many were caught; F1 = harmonic mean of the two.
- **ROC-AUC** -- ranking quality of the continuous score, independent of any one threshold. The project explicitly notes that its [-1,+1] circuit output, affine-rescaled to [0,1] for this computation, is **not a calibrated probability** -- just a monotonic score for ranking.
- **Cost-sensitive mean cost** -- average cost per prediction using the real UCI cost matrix (Section 5.5), the single number closest to 'real-world financial impact'.

13.3 Trainability metrics

- **Gradient norm** (raw and parameter-count-normalized) -- the normalized version divides by sqrt(number of active trainable parameters) so that Direct Training (gradient over the whole L-layer tensor) and Layer-wise Training (gradient over just 1 layer at a time) can be compared on equal footing, per-parameter.
- **Plateau fraction** -- the fraction of recorded updates whose gradient norm fell below a predeclared threshold (0.05) -- a direct, numeric trainability indicator.
- **Convergence / success rate** -- a run 'converges' if validation balanced accuracy reaches a predeclared target (the classical logistic-regression baseline's validation balanced accuracy) and stays there for 3 consecutive checkpoints; a run that never does is honestly marked 'not converged', never assigned a fabricated convergence time.

13.4 Statistical comparison

For every (qubit count, depth) cell, the paired difference (Layer-wise minus Direct) across matched seeds is tested with the **Wilcoxon signed-rank test** when ≥6 paired seeds exist, else a **two-sided sign-flip permutation test** (10,000 permutations) -- this choice was fixed **before** running the experiment, not picked afterwards to get a favourable p-value. A 95% bootstrap confidence interval on the mean paired difference is always reported alongside.

14. Classical Baselines

Two classical models provide context so the VQC is never evaluated in isolation: **Logistic Regression** and a small one-hidden-layer (16 units) **MLP**. Each is evaluated on **two** input representations, and which one is used is always stated explicitly:

- **Full features** -- all 61 one-hot + scaled columns (the standard classical representation, no PCA).
- **PCA-4d** -- the exact same 4-dimensional PCA representation the 4-qubit VQC receives, giving a fair, matched-dimensionality comparison point.

Both are fit using training+validation data only; the test set is touched once, for final reporting, exactly like the VQC.

15. Results (Actual Numbers, Quick Mode)

Every number below comes directly from the saved CSV tables / Excel workbook in tables/, generated by code from results/quick/*.csv -- nothing here is typed by hand into a report.

15.1 Dataset and preprocessing (Table 1)

1000 instances, 20 raw features, 0 missing values, 700/150/150 train/val/test. PCA-to-4 explains 41.4% of variance; PCA-to-6 explains 53.8%.

15.2 Classical baselines (Table 3)

Model	Features	Test balanced acc.	Test F1	Test ROC-AUC	Mean cost
Logistic Regression	Full (61-d)	0.690	0.568	0.780	0.753
Logistic Regression	PCA-4d	0.557	0.226	0.691	1.313
MLP (16 units)	Full (61-d)	0.545	0.224	0.666	1.324
MLP (16 units)	PCA-4d	0.543	0.197	0.612	1.333

Logistic Regression on the full feature set is the strongest model overall by a wide margin. Note how far it drops (0.690 → 0.557 balanced accuracy) when restricted to the same 4-dimensional PCA input the VQC sees -- a classical model loses most of its edge once it is given the same limited information as the quantum circuit.

15.3 Main VQC grid, test-set results (Table 4, selected rows)

Qubits	Depth	Protocol	Test bal. acc.	Test F1
4	2	Direct	0.496	0.109
4	2	Layer-wise	0.588	0.298
4	12	Direct	0.548	0.193
4	12	Layer-wise	0.588	0.325
6	6	Direct	0.531	0.137
6	6	Layer-wise	0.600	0.341
6	12	Direct	0.523	0.127
6	12	Layer-wise	0.593	0.330

Across all 6 (qubit, depth) cells tested: Layer-wise beat Direct on test balanced accuracy in 5 of 6 (the sixth, 6 qubits/depth 2, was a near-tie, 0.591 vs 0.590), and beat Direct on test F1 in

all 6.

15.4 The accuracy-vs-balanced-accuracy paradox (the most important single finding to be able to explain)

Direct Training's validation-set **raw accuracy** curve climbs quickly to ~0.70 and plateaus there -- but its **validation balanced accuracy** stays near 0.51 (chance level) the entire time. This is the exact signature of a model collapsing toward always predicting the majority class ('good credit'), which, given the 70/30 split, trivially yields ~70% raw accuracy while discriminating almost nothing. Layer-wise Training's raw-accuracy curve settles lower (~0.60), but its balanced accuracy and F1 are meaningfully higher, meaning it is actually distinguishing some bad-credit applicants that Direct Training's degenerate solution misses entirely. **This project treats balanced accuracy/F1, not raw accuracy, as the metric that matters on this imbalanced task** -- so this result is read as evidence **favouring** Layer-wise Training's trainability, even though Direct 'wins' on the naive number.

15.5 Convergence and trainability (Table 5)

- **Success rate = 0.0 for every single cell, both protocols.** Neither protocol reached the predeclared convergence target (classical-baseline validation balanced accuracy, 0.7397) within the 150-update quick-mode budget. Stated plainly: quick mode demonstrates and compares trainability, it does not claim either protocol 'solved' the task.
- Direct Training's **normalized gradient norm at initialization** decreases monotonically with depth (0.220 at depth 2 → 0.102 at depth 12, for 4 qubits) -- a barren-plateau signature, consistent with the separate diagnostic.
- Layer-wise Training's recorded 'at-init' gradient is, by construction, always the first single-layer stage's gradient, so it stays constant across target depths within a qubit count (0.188 for 4 qubits) -- this constancy is the intended mechanism of the method (every run starts from the same shallow, well-conditioned landscape), not a measurement artifact.
- **Plateau fraction is HIGHER for Layer-wise (0.23-0.34) than Direct (~0.00-0.002)** across every cell -- a genuinely counter-intuitive result, explained in Section 16.3; it is reported honestly rather than hidden because it complicates a simple 'layer-wise always wins' story.

15.6 Sampling noise and depolarizing noise (Tables 7 and 8)

Finite shots (100, 1000) moved test balanced accuracy by only a few percentage points relative to exact/analytic at the tested configuration, and Layer-wise stayed ahead of Direct at every shot setting tested. Depolarizing noise at the tested depth (2) and probabilities (0, 0.001, 0.01) caused **no change to three decimal places** in discrete test balanced accuracy for either protocol -- verified as a genuine finding (continuous MSE loss did shift by ~1e-4 to 1e-3, just not enough to flip any individual prediction at this shallow depth and low noise level), not a bug, and consistent with Wang et al. (2021)'s result that noise-induced degradation needs depth to accumulate.

15.7 Gradient-variance diagnostic (circuit-only, Section 3.5 numbers restated)

Gradient-component variance falls from ~0.49-0.52 at depth 1 to 0.032 (4 qubits), 0.0081 (6 qubits), 0.0019 (8 qubits) at depth 24. A log-linear (exponential-in-depth) fit gives $R^2 = 0.73$ (4 qubits), 0.82 (6 qubits), 0.89 (8 qubits) -- a reasonably good but visibly imperfect fit, since the curve flattens above depth ~16 for every qubit count. The honest description is 'approximately exponential over the tested range, with a floor at high depth', not an unqualified 'exponential decay'.

15.8 Layer-insertion loss (the architectural-caveat evidence, Section 7.3)

Measured directly (not estimated) in every real layer-wise run: inserting a new near-zero-angle layer changes the validation loss by a small, non-zero amount (e.g. +0.0010, -0.0001, +0.0008 across successive insertions in one real 4-qubit/depth-12 run) before any training of that stage occurs -- confirming the caveat that a near-zero layer is not an identity operation, because its CZ gates stay active regardless of the rotation angles.

15.9 Wall-clock time (Table 5 / figures/wallclock_comparison.png)

Layer-wise Training was **consistently and substantially faster** than Direct Training at every tested depth: e.g. at depth 12 (averaged over 4 and 6 qubits), Direct averaged ~17.6 seconds per run versus Layer-wise's ~7.3 seconds -- roughly 2.4× faster, because early layer-wise stages only execute a shallow (cheap) circuit on average across the whole run.

15.10 Statistical significance (Table 9)

Across all 18 (qubit, depth, metric) paired comparisons, **no p-value is below 0.05** (the permutation test was used throughout, since 3 paired seeds < the predeclared Wilcoxon threshold of 6); most p-values are ≥ 0.495, and every 95% confidence interval on the paired difference crosses zero. The point estimates consistently favour Layer-wise on balanced accuracy (positive difference in 5 of 6 cells) and F1 (positive in all 6), but with only 3 seeds **this project does not claim statistical significance it did not measure** -- this is explicitly stated as a limitation, not glossed over.

15.11 Confusion matrices (best run per protocol, and the two headline classical baselines)

For the single best-performing saved run of each protocol (highest test balanced accuracy, picked programmatically from the real results, not by eye): the best Layer-wise run (4 qubits, depth 12, seed 0, test balanced accuracy 0.651) classified the 150-example test set as 69 true negatives (correctly 'good'), 36 false positives, 16 false negatives, and 29 true positives (correctly 'bad') -- i.e. it correctly caught 29 of 45 actually-bad-credit applicants (recall ~0.64). These, and the matching confusion matrices for Logistic Regression on full features and on the matching 4-qubit PCA representation, are generated by re-loading each run's exact saved weights and re-evaluating on the untouched test set (src/plotting.py's plot_confusion_matrix, wired up in run_project.py) -- see Section 20.12-20.13 for the full figure-by-figure description.

16. Analysis and Honest Interpretation

16.1 Which settings favoured Layer-wise Training?

Test balanced accuracy and F1 (5-6 of 6 cells) and wall-clock speed (6 of 6 cells), fairly consistently across both tested qubit counts and all three tested depths -- some evidence the effect is not a fluke of one configuration, though not statistically confirmed at 3 seeds.

16.2 Which settings favoured Direct Training?

Raw test accuracy (every cell) and validation balanced accuracy (every cell) -- both explained by the majority-class-collapse behaviour (Section 15.4), which this project treats as evidence against, not for, Direct Training's actual trainability on this imbalanced task.

16.3 Reconciling the plateau-fraction result with the overall 'Layer-wise helps' story

Layer-wise Training spends a **larger** share of updates in a low-gradient state (0.23-0.34) than Direct (~0.00-0.002) -- this is not a contradiction once you note that Layer-wise deliberately resets to a near-zero-angle initialization at the start of every new stage after the first, and that near-zero region genuinely does have small gradients while the new layer 'warms up' (visible directly in the layer-insertion loss jumps, Section 15.8). So Layer-wise trades a larger share of small-gradient **individual updates** for a better-conditioned **overall landscape at the start of each stage** -- two different, both-legitimate senses of 'plateau', reported separately rather than collapsed into one misleading number.

16.4 Did depth or noise change the relative picture?

The Direct-vs-Layer-wise pattern held at every tested depth (2, 6, 12) for both qubit counts -- it did not reverse within the tested range. Finite shots and the tested noise levels did not materially change either protocol's relative standing, but both were only tested on a narrow, predeclared subset (Section 12.3), so no claim is made about deeper or noisier regimes that were not actually run.

16.5 Was the classical baseline stronger?

Yes, substantially: full-feature Logistic Regression (0.690 balanced accuracy) beat every VQC configuration (0.50-0.60) and also beat Logistic Regression restricted to the matching 4-qubit PCA input (0.557) -- though the VQCs, especially Layer-wise, generally beat that equivalently-reduced classical baseline, suggesting the quantum circuit extracts somewhat more from the same 4 numbers than a linear classical model does, even though neither approaches the full-feature classical result.

16.6 Bottom-line, exam-ready summary sentence

"Within the quick-mode budget we actually ran, Layer-wise Training showed a consistent but not-yet-statistically-significant advantage on the metrics that matter for this imbalanced task (balanced accuracy, F1) and was clearly faster, while our own gradient-variance diagnostic confirms this architecture genuinely exhibits a barren-plateau effect that gets worse with depth and qubit count -- consistent with, but not proof of, the mechanism Layer-wise Training is designed to exploit; neither protocol solved the task outright, and most of the gap to classical performance traces to the unavoidable information loss from compressing 61 features down to 4-8 PCA dimensions, not to the quantum model itself."

17. Limitations (List These Proactively, Don't Wait to Be Asked)

- Only 3 random seeds per cell in quick mode → limited statistical power; the full-mode plan uses 10.
- Small (1000-row), imbalanced (700/300), feature-coarse dataset.
- PCA retains only 41-54% of processed feature variance.
- A single fixed ansatz (RY+RZ per qubit per layer, CZ ring, single-qubit PauliZ readout) was tested; results do not automatically generalize to other circuit designs.
- Noise and shots sub-experiments used a reduced, predeclared subset (4 qubits only; limited depths) for measured computational feasibility.
- Classical simulators only (default.qubit, default.mixed) -- no results describe real quantum hardware.
- 8-qubit depth was only tested in the circuit-only gradient diagnostic, never trained on the actual dataset in quick mode.
- Neither protocol converged to the predeclared target within the quick-mode budget.
- The full experimental grid (10 seeds, depths to 24, 8 qubits, full shots/noise grid) was implemented, configured, and is resumable, but was not exhaustively executed for this delivery -- pilot-estimated at 3.5-4 hours of CPU time.

18. The Repository, File by File

GitHub: github.com/Tharungk2005/Barren_plateau-in-VQC. This section goes through every file that matters, in the order you would naturally open them, with its purpose, its most important code lines, and what to say about each. Folder structure:

```
vqc_layerwise_credit_risk/ config.json run_project.py requirements.txt README.md
src/ tests/ data/raw data/processed results/quick results/full tables/ figures/ notebooks/ reports/
```

18.1 config.json -- the single source of truth

Every number that controls an experiment lives here: qubit counts, depths, seeds, update budget, learning rate, batch size, the label convention (-1/+1), the official UCI cost matrix, shot settings, noise probabilities, and the statistical-test choice. Nothing in src/ hardcodes an experimental setting -- every module reads it from the cfg dictionary passed into it. This is why the project can honestly claim its experiments were 'predeclared' (Section 4.5): you can open this one file and see every choice made before any run started.

Key lines

```
"experiment_grid": {"quick": {"qubit_counts": [4,6], "depths": [2,6,12],
"protocols": ["direct","layerwise"], "seeds": [0,1,2]}}
"label_convention": {"good_credit_encoded_label": -1, "bad_credit_encoded_label": 1}
```

18.2 run_project.py -- the one command that does everything

A single script, run with --mode quick or --mode full, that executes the whole pipeline in 9 steps: fetch/validate dataset -> preprocess per qubit count -> classical baselines -> main VQC grid -> shots experiment -> noise experiment -> gradient diagnostic -> circuit diagrams -> tables/figures. It imports every src/ module and calls them in order; it contains no scientific logic of its own, only orchestration -- this is deliberate, so the exact same tested functions run whether you call them from this script, from a test, or from the notebook.

Key lines (the 9-step structure)

```
df = dr.fetch_raw_dataset(cfg) # Step 1
prepared = pp.prepare_dataset_for_qubits(df, y, nq, cfg, seed=0) # Step 2
main_csv = ER.run_main_grid(cfg, df, y, convergence_target) # Step 4
generate_confusion_matrices(main_df, prepared_by_qubits, ...) # confusion matrices
```

The generate_confusion_matrices() function (added after the confusion-matrix gap was found during review -- Section 20.9) finds the single best-performing saved run for each protocol by test_balanced_accuracy, reloads its exact trained weights from the saved JSON history, re-evaluates them on the untouched test set, and plots the result -- it does not retrain anything.

18.3 src/utils.py -- shared plumbing

- **load_config()** -- reads config.json into a plain dict.
- **set_global_seed(seed)** -- seeds Python's random and NumPy's global RNG for reproducibility.
- **environment_snapshot()** -- captures exact installed package versions (numpy, scipy, pandas, scikit-learn, matplotlib, pennylane) and Python version into every result manifest, so a run's environment is always recorded, not just assumed.
- **save_json / load_json** -- JSON read/write with a custom encoder that handles NumPy int/float/array types (which plain json.dump cannot serialize directly).

18.4 src/data_retrieval.py -- real dataset retrieval and validation

Fetches the UCI German Credit dataset via ucimlrepo.fetch_ucirepo(id=144), caches it to data/raw/german_credit_raw.csv, and validates its schema before anything else runs.

Key lines

```
repo = fetch_ucirepo(id=cfg["data"]["uci_dataset_id"])
UCI_COST_MATRIX = {("good","bad"): 1, ("bad","good"): 5, ...} # transcribed verbatim
```

- **validate_schema()** -- checks row count == 1000, feature count == 20, target classes subset of {1,2}, and zero missing values; raises DatasetRetrievalError with explicit recovery instructions (never silently substitutes another dataset) if anything is off.
- **DatasetRetrievalError** -- a custom exception whose message tells the user exactly how to recover (check connection, retry, manually place a CSV at the documented path) rather than crashing opaquely.

18.5 src/preprocessing.py -- leakage-safe pipeline

Key lines

```
X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.30,
random_state=seed, stratify=y) # then split X_temp 50/50 -> val, test
pipeline = Pipeline([("pre", ColumnTransformer([...])), ("pca", PCA(n_components=n_qubits))])
pipeline.fit(X_train) # fit ONLY on training data -- the leakage-prevention line
```

- **AngleScaler** class -- a small custom min-max scaler to [0,π]; its .fit() stores only the training split's min_/max_; its .transform() clips out-of-range validation/test values into [0,π] instead of silently producing invalid rotation angles.
- The three disjoint-index asserts in stratified_splits() (assert not (idx_train & idx_val), etc.) are what the test suite's test_no_index_overlap checks externally too -- belt and suspenders.

18.6 src/circuits.py -- the quantum circuit itself

The single place the VQC architecture is defined; both training protocols import from here, so neither can silently diverge from the other's circuit structure.

Key lines

```
def entangling_edges(n_qubits):
if n_qubits == 2: return [(0, 1)] # avoid double-applying the one ring edge
return [(i, (i+1) % n_qubits) for i in range(n_qubits)]

def n_trainable_params(n_qubits, n_layers):
return 2 * n_qubits * n_layers

for i in range(n_qubits): qml.RY(x[..., i], wires=i) # encoding
for i in range(n_qubits):
qml.RY(wl[i,0], wires=i); qml.RZ(wl[i,1], wires=i) # trainable rotation
for (a,b) in entangling_edges(n_qubits): qml.CZ(wires=[a,b]) # fixed entangler
```

The x[..., i] indexing (not x[i]) is deliberate -- it is what makes the same circuit function work both for a single sample (x shape (n_qubits,)) and for a whole batch at once (x shape (batch, n_qubits)), letting PennyLane's parameter broadcasting train on a batch in one circuit call instead of a slow Python loop per sample. (This indexing bug -- using x[i] instead of x[...,i] -- was one of the first things caught and fixed while building the training loop; see Section 20.9.)

18.7 src/training.py -- both training protocols

Key lines: the manual Adam optimizer

```
self.m = self.beta1*self.m + (1-self.beta1)*grad # 1st moment (momentum)
self.v = self.beta2*self.v + (1-self.beta2)*(grad**2) # 2nd moment
return params - self.lr * mhat / (np.sqrt(vhat) + self.eps)
```

Key lines: Direct Training (whole tensor is trainable)

```
weights = pnp.array(C.init_weights_uniform(n_qubits, n_layers, 0, 2*pi, rng), requires_grad=True)
grad = qml.grad(cost, argnums=0)(weights, Xb, yb) # gradient over ALL layers at once
```

Key lines: Layer-wise Training (freezing, the most important code in the project)

```
active = pnp.array(new_layer, requires_grad=True) # only the NEW layer is differentiable
frozen_const = pnp.array(frozen_weights, requires_grad=False) # old layers: plain constant

def cost(active_w, frozen_w, Xb, yb):
full_w = pnp.concatenate([frozen_w, active_w], axis=0) # old layers still AFFECT the output...
preds = qnode(Xb, full_w)
return pnp.mean((preds - yb)**2)

grad_fn = qml.grad(cost, argnums=0) # ...but gradient is taken w.r.t. active_w ONLY
```

This is the single most important code block to be able to explain in the whole project: the old layers are concatenated into the forward pass (so they still shape the circuit's output), but argnums=0 tells qml.grad to differentiate only with respect to active_w -- the frozen layers mathematically cannot appear in the gradient, so Adam's update step, which only ever sees that gradient, cannot move them.

Key lines: the layer-insertion-loss measurement (the architectural-caveat evidence)

```
preds_prior = _evaluate_full(qnode, full_prior, X_val, y_val, n_layers_active=layer_idx)
loss_prior_stage_end = _cost_mean_sq(preds_prior, y_val)
preds_after_insertion = _evaluate_full(qnode, full_prior, X_val, y_val, n_layers_active=layer_idx+1)
loss_immediately_after_insertion = _cost_mean_sq(preds_after_insertion, y_val)
```

Same weights (full_prior), evaluated with one more layer switched on -- isolating exactly the effect of the new layer's CZ gates before any training touches it.

Key lines: the freeze-integrity check

```
frozen_before_stage = np.array(frozen_const).copy()
# ... training loop runs ...
frozen_unchanged = bool(np.array_equal(frozen_before_stage, np.array(frozen_const)))
```

An exact (not approximate) NumPy array-equality check, logged into stage_logs['frozen_params_unchanged'] for every single stage of every real run -- not just asserted in a test, but verified in the actual saved results.

18.8 src/metrics.py -- every evaluation number

Key lines

```
def predict_labels(outputs, threshold=0.0):
return np.where(np.asarray(outputs) > threshold, 1, -1)

def continuous_score(outputs): # for ROC-AUC only, NOT a probability
return (np.asarray(outputs, dtype=float) + 1.0) / 2.0

def normalized_gradient_norm(grad_array):
return float(np.linalg.norm(g) / np.sqrt(g.size)) # per-parameter comparability
```

misclassification_cost() looks up each (actual, predicted) pair in the UCI cost matrix dict and sums it -- this is where the 5x-worse-to-miss-bad-credit rule from Section 5.5 actually gets applied to real predictions.

18.9 src/baselines.py -- classical comparison models

Trains Logistic Regression and a small MLP on two representations (full 61 features; the same n_qubits-dimensional PCA the VQC sees) and reuses the exact same metrics.compute_classification_metrics function the VQC results use, by rescaling sklearn's predict_proba output into the same [-1,+1] pseudo-circuit-output range first -- so classical and quantum results are computed by identical code, not two different scoring paths that might subtly disagree.

Key line

```
def _scores_to_pm1_outputs(scores_pos_class):
return 2.0 * np.asarray(scores_pos_class) - 1.0 # [0,1] probability -> [-1,+1] pseudo-output
```

18.10 src/gradient_diagnostics.py -- the barren-plateau diagnostic

Key lines

```
w = pnp.array(rng.uniform(0, 2*np.pi, size=(n_layers, n_qubits, 2)), requires_grad=True)
g = qml.grad(qnode, argnums=1)(x, w)
grad_component[s] = g_np[0, 0, 0] # one representative parameter's gradient
grad_norms[s] = np.linalg.norm(g_np) # full gradient-vector norm, for comparison
```

Repeated n_samples times (60 in quick mode) per (qubit count, depth) cell, with a fixed encoding input (x = pi/4 on every qubit) so only the ansatz parameters vary -- isolating the ansatz's own trainability from any dataset effect. bootstrap_variance_ci() then resamples those samples 2000 times to get a 95% confidence interval on the variance itself, not just a point estimate.

18.11 src/statistics_utils.py -- the predeclared statistical test

Key lines

```
if n >= 6 and np.any(diffs != 0):
stat, p_value = stats.wilcoxon(diffs); test_used = "wilcoxon"
else:
test_used = cfg["statistics"]["fallback_test"] # permutation, predeclared
```

The n>=6 threshold and the fallback choice are read from config.json, decided before any result was seen -- with only 3 seeds in quick mode, every cell actually used the permutation-test branch (Section 15.10).

18.12 src/experiment_runner.py -- orchestration, resumability, failure handling

Key lines: resumability

```
def _load_existing_run_ids(csv_path):
df = pd.read_csv(csv_path)
return set(df[df["status"] == "success"]["run_id"].tolist())

if run_id in existing: continue # skip -- already done successfully
```

Key lines: a run can fail without crashing the whole grid

```
try:
res = T.direct_training_run(...)
... # compute metrics, build the success row
except Exception as exc:
row = {... "status": "failed", "error_message": f"{exc}\n{traceback.format_exc()}"}
return row, False
```

Every run -- success or failure -- is appended to the CSV immediately (_append_row), so a crash partway through a multi-hour full-mode run loses at most the one in-progress run, never anything already completed.

18.13 src/plotting.py -- every figure, generated from saved CSVs only

No function in this file ever trains anything or reads from a live model -- every plot function's only inputs are a pandas DataFrame (read from a results CSV) and/or the saved JSON training histories. This is what makes every figure "regeneratable": delete figures/, run the plotting step again, get the same figures back from the same saved numbers.

Key line (a real bug found and fixed during review)

```
# WRONG (silently used only the first qubit count's row):
# ys = [sub[sub.n_layers==d]["total_wall_clock_s"].values[0] ...]
# FIXED -- average across qubit counts first, THEN average across seeds:
per_qubit = df.groupby(["n_qubits","n_layers","protocol"])[metric].mean().reset_index()
agg = per_qubit.groupby(["n_layers","protocol"])[metric].mean().reset_index()
```

18.14 src/export.py -- the 9 required tables and the Excel workbook

Key line (another real bug found and fixed)

```
# pandas groupby silently DROPS rows whose key is NaN -- "analytic" (no-shots) runs
# store shots as NaN, so they vanished from Table 7 until this fix:
success["shots"] = success["shots"].apply(lambda v: "analytic" if pd.isna(v) else int(v))
```

save_excel_workbook() writes one sheet per table into tables/results_summary.xlsx, rounding floats to 4 decimals only for display -- the underlying CSVs keep full precision for anyone who wants to redo the statistics.

18.15 tests/ -- the 45-test automated suite

- **test_data_retrieval.py** -- real dataset schema, class counts match documentation, bad-schema rejection.
- **test_preprocessing.py** -- label mapping, no train/val/test index overlap, correct input dimension per qubit count, angles within [0,π], and the leakage proof (PCA fit on different training subsets gives different components).
- **test_circuits.py** -- the 2-qubit ring doesn't duplicate its one edge, the p=2nL formula, output bounded in [-1,1], finite gradients, batched output matches looping over single samples one at a time.
- **test_training.py** -- exact reproducibility under a fixed seed, parameters actually move during training, finite loss/gradients, **frozen layers provably unchanged**, trainable-parameter count grows correctly stage by stage, the corrected layer-insertion-loss regression test.

- **test_metrics.py** -- known-value checks (e.g. a hand-computed MSE), the cost-matrix asymmetry, perfect-prediction metrics equal 1.0.
- **test_experiment_runner.py** -- a simulated training failure is caught and logged as 'failed' without crashing the grid; re-running an already-completed grid adds zero new rows.
- **test_export.py / test_plotting.py** -- the analytic-shots-row fix and the qubit-averaging fix are both locked in as regression tests so they cannot silently reappear.
- **test_baselines.py, test_gradient_diagnostics.py, test_statistics_utils.py** -- baseline output ranges, gradient-sample shapes and the exponential-fit sanity checks, and the statistical-test pipeline.

18.16 data/, results/, notebooks/, reports/

- **data/raw/** -- german_credit_raw.csv (the cached dataset) and german_credit_metadata.json (source URL, DOI, retrieval timestamp, class counts, the cost matrix).
- **data/processed/** -- pipeline_4q.joblib / pipeline_6q.joblib (the fitted ColumnTransformer+PCA+AngleScaler objects, pickled) and preprocessing_metadata.json (split sizes, explained variance).
- **results/quick/*.csv** -- one row per run (run_id, every metric, status, timing); **results/quick/histories/*.json** -- per-run full training curves and final weights, keyed by the same run_id; **result_manifest.json** -- a summary of how many rows succeeded/failed per file.
- **notebooks/vqc_layerwise_project.ipynb** -- imports the same src/ modules (no duplicated logic), demonstrates every pipeline stage live plus loads the saved quick-mode results; has been executed end-to-end with real outputs saved in it.
- **reports/final_report.md** -- the full written report (20 sections: abstract through references) this study guide is distilled from. **reports/viva_study_guide.pdf** -- this document.

19. Every Table Explained, Column by Column

All in tables/ as individual CSVs plus one combined tables/results_summary.xlsx workbook. In the order a professor would naturally ask to see them:

19.1 table1_dataset_summary.csv

One row per qubit count. Columns: dataset_source, n_instances (1000), n_raw_features (20), class_counts, n_qubits_representation, n_train/n_val/n_test (700/150/150), pca_explained_variance_ratio_sum (0.414 for 4 qubits, 0.538 for 6), missing_values.

19.2 table2_experimental_configs.csv

One row per (qubit count, depth) cell actually run. Columns include n_trainable_params (computed from 2nL, not typed in), ansatz description, optimizer/learning_rate/batch_size, seeds, total_update_budget, the two protocols' initialization schemes, gradient_method, shots, noise.

19.3 table3_classical_baselines.csv

One row per (model, feature representation): Logistic Regression and MLP, each on full-features and PCA-4d, with val/test accuracy, balanced accuracy, precision, recall, F1, ROC-AUC, mean cost, averaged over the seeds in n_seeds.

19.4 table4_main_vqc_results.csv

The central results table: one row per (n_qubits, n_layers, protocol), aggregated (mean and std) across the 3 seeds. Columns: n_seeds, val/test accuracy and balanced accuracy (mean+std), precision, recall, F1 (mean+std), ROC-AUC, loss, mean cost, and n_failed (always 0 here).

19.5 table5_convergence_trainability.csv

One row per (n_qubits, n_layers, protocol): success_rate (fraction of seeds that reached the convergence target -- 0.0 everywhere in this run), mean_convergence_update, grad_norm_at_init_mean / grad_norm_at_init_normalized_mean, plateau_fraction_mean, mean_wall_clock_s, mean_circuit_evaluations.

19.6 table6_seed_level_detail.csv and table6_seed_level_summary.csv

detail has one row per individual seed (no averaging -- every run's test_balanced_accuracy, F1, ROC-AUC, wall-clock time, so nothing is hidden behind an average); summary has the same grouped by cell with _mean/_std suffixes (flattened from a pandas multi-index, which is exactly the bug that originally crashed the Excel export -- Section 20.9).

19.7 table7_shots_results.csv

One row per (n_qubits, n_layers, protocol, shots), where shots is 'analytic', 1000, or 100 -- balanced accuracy and F1 at each sampling regime.

19.8 table8_noise_results.csv

One row per (n_qubits, n_layers, protocol, noise_prob), where noise_prob is 0.0, 0.001, or 0.01.

19.9 table9_statistical_comparison.csv

One row per (n_qubits, n_layers, metric) -- 18 rows (6 cells × 3 metrics: balanced accuracy, F1, ROC-AUC). Columns: n_paired_seeds, mean_paired_difference_layerwise_minus_direct, ci_lower, ci_upper, confidence_level, test_used, statistic, p_value.

19.10 gradient_diagnostic_summary.csv

One row per (n_qubits, n_layers) in the circuit-only diagnostic (24 rows: 3 qubit counts × 8 depths). Columns: n_samples, gradient_component_variance (+ its 95% CI lo/hi), gradient_component_mean, grad_norm_mean, grad_norm_std, init_distribution, gradient_method, fixed_input.

19.11 classical_baselines_raw.csv

The per-seed (not aggregated) classical-baseline results that table3 is built from.

20. Every Figure Explained, Type by Type

All in figures/, each saved as both .png and .pdf. 120 files, built from several repeated figure 'types' across different (qubit count, depth, seed) combinations -- explained once per type below, in the order you would walk through them.

20.1 circuit_diagram_nq{Q}_d{D}.png -- 6 files

The actual executed circuit, drawn directly from the same PennyLane QNode the experiments ran (not a hand-drawn illustration): encoding RY gates, then per-layer RY/RZ/CZ blocks, then the measurement symbol on wire 0. For depth 12 (too many layers to stay legible), only the first 3 layers are drawn and the figure is explicitly labeled '...first 3 layers of 12' -- a documented legibility trade-off, not a hidden truncation.

20.2 val_accuracy_vs_updates_nq{Q}_d{D}.png -- 6 files (THE key figure)

X-axis: optimizer update number. Y-axis: validation accuracy. Two lines (Direct in red, Layer-wise in blue), each the mean across 3 seeds with a shaded +/-1 std-dev band, legend states n=3 seeds. This is the figure that reveals the accuracy-vs-balanced-accuracy paradox (Section 15.4): Direct's curve rises fast and flattens near 0.70 (majority-class collapse); Layer-wise's curve settles lower but represents genuine discrimination once you cross-reference Table 4's balanced accuracy.

20.3 train_loss_nq{Q}_d{D}.png and val_loss_nq{Q}_d{D}.png -- 12 files

Same layout, Y-axis is MSE loss instead of accuracy -- shows both protocols' raw optimization curves (not accuracy-derived), useful to point out that Layer-wise's curve is visibly piecewise (small steps/jumps at each layer insertion) while Direct's is smooth from the start.

20.4 grad_norm_nq{Q}_d{D}.png -- 6 files

Raw gradient-vector norm over the course of training, same two-line format. Complements the normalized version reported in Table 5 (not separately plotted, since the normalized metric is only meaningful as a single end-of-training number, not a full curve).

20.5 gradient_variance_diagnostic.png -- 1 file (the barren-plateau evidence figure)

X-axis: circuit depth (1-24, log-scale not used on X). Y-axis: gradient-component variance, log-scale. Three lines (4, 6, 8 qubits), each point with a 95% bootstrap confidence-interval error bar, n_samples=60 stated in the title. This is circuit-only -- no dataset involved -- and is the project's single strongest piece of direct evidence that a barren plateau exists in this exact architecture (Section 3.5).

20.6 layerwise_stage_performance_nq{Q}_d{D}_seed{S}.png -- 18 files

X-axis: number of layers active (1 to target depth). Blue line with circles: validation loss after that stage finished training. Red X markers: validation loss immediately after that layer was inserted (near-zero angles) but **before** any training update of that stage -- this is the direct, per-run evidence for the architectural caveat (Section 7.3): the X markers sit a small but non-zero distance from the previous stage's final loss, proving the new layer is not an identity operation.

20.7 success_rate_comparison.png -- 1 file

Bar chart, X-axis = depth, grouped bars = protocol, Y-axis = fraction of seeds that reached the convergence target, correctly averaged across both tested qubit counts (a two-stage groupby -- first per qubit count, then across qubit counts -- after the earlier single-qubit-count bug was fixed). In this run every bar is 0.0 (Table 5, Section 15.5).

20.8 final_metric_comparison_test_balanced_accuracy.png and final_metric_comparison_test_f1.png -- 2 files

Grouped bar charts, one cluster of bars per qubit count, bars within a cluster = depth × protocol, error bars = std across seeds. The headline 'who-won-each-cell' figure for Table 4.

20.9 wallclock_comparison.png -- 1 file

Bar chart, X-axis = depth, grouped bars = protocol, Y-axis = mean wall-clock seconds, correctly averaged across both qubit counts (same two-stage-groupby fix as 20.7). Shows Layer-wise consistently faster at every depth (Section 15.9).

20.10 shots_sensitivity_nq4_d{2,6}.png -- 2 files

X-axis: shot setting (analytic, 1000, 100). Y-axis: test balanced accuracy, mean+std across the tested seeds. Shows both protocols are only mildly sensitive to finite sampling at this configuration.

20.11 noise_sensitivity_nq4_d2.png -- 1 file

X-axis: depolarizing probability (0, 0.001, 0.01). Y-axis: test balanced accuracy. Essentially flat lines for both protocols at this shallow depth/low-noise regime (Section 15.6) -- a real, verified result, not a plotting artifact.

20.12 confusion_matrix_vqc_direct_best.png and confusion_matrix_vqc_layerwise_best.png -- 2 files

2x2 heatmaps (rows = actual, columns = predicted; Good credit / Bad credit), counts annotated in each cell, for the single best-performing saved run of each protocol (by test balanced accuracy, picked automatically from the real results, not cherry-picked by eye). Title states the exact configuration (qubits, depth, seed) and its test balanced accuracy.

20.13 confusion_matrix_logreg_full.png and confusion_matrix_logreg_pca.png -- 2 files

The same style of confusion matrix for the two headline classical baselines (Logistic Regression on full features, and on the matching 4-qubit PCA representation), so the quantum and classical confusion matrices can be compared cell-by-cell on the same test set.

21. Live Demonstration Script

Suggested order for a live demo, each step mapped to what to say:

1. **Show config.json** -- "every experimental setting lives in one place, nothing is hardcoded inside the training code."
2. **Run** *python run_project.py --mode quick* **from a clean clone** (or show the already-completed run_summary.json) -- "this single command re-downloads and validates the real dataset, trains both protocols across the whole grid, runs the noise/shots/gradient-diagnostic experiments, and regenerates every table and figure, in under 6 minutes."
3. **Open a circuit diagram** (figures/circuit_diagram_nq4_d2.png) -- walk through encoding RY gates, the RY/RZ/CZ layer structure, and the single-qubit measurement, matching Section 7 exactly.
4. **Open figures/val_accuracy_vs_updates_nq4_d12.png** -- point out Direct's high-but-flat raw-accuracy curve versus Layer-wise's lower-but-more-meaningful curve; this is the single best figure to anchor the whole 'accuracy vs balanced accuracy' story (Section 15.4).
5. **Open figures/gradient_variance_diagnostic.png** -- the direct, dataset-independent evidence of the barren plateau (Section 3.5 / 15.7).
6. **Open figures/layerwise_stage_performance_nq4_d12_seed0.png** -- the architectural-caveat evidence (Section 7.3 / 15.8): point out the small jump at every 'x' marker.
7. **Open tables/results_summary.xlsx** -- show Table 9 (statistics) to demonstrate the project reports non-significant results honestly rather than hiding them.
8. **Run** *pytest tests/ -v* -- "45 automated tests, including a direct check that frozen parameters in layer-wise training literally cannot change."
9. **Close with the one-sentence bottom line** (Section 16.6).

22. Suggested Division Among 3 Team Members (for the live talk only -- everyone must still know all of it for Q&A;)

Person	Primary speaking sections	Must still be ready for
A	Sections 1-4, 7 (quantum basics, barren plateaus, circuit architecture, research questions)	Any basic quantum-computing or circuit-design question
B	Sections 5-6, 9-12 (dataset, preprocessing, both training protocols, experimental design)	Any question about the training algorithms, leakage, or experiment scope
C	Sections 13-21 (metrics, results, analysis, limitations, repository/table/figure walkthrough, demo)	Any question about a specific number, figure, code line, or statistical claim

An examiner can and often will ask any team member a question from any section -- the division above is only for who leads each part of the spoken explanation, not a reason to skip

studying the rest of this document.

23. Anticipated Viva Questions and Model Answers

Study every answer below until you could give it without notes. Answers are written in the same technical, non-analogy style the rest of this document uses.

Q: What is a qubit, in one precise sentence?

A: A qubit is a two-level quantum system whose state is a normalized complex vector $\alpha|0\rangle+\beta|1\rangle$ with $|\alpha|^2+|\beta|^2=1$; measuring it collapses it probabilistically to $|0\rangle$ or $|1\rangle$.

Q: Why do you use RY and RZ specifically, and not, say, RX or a general U3 gate?

A: RY encodes a real classical number as a real rotation (clean, monotonic mapping for the encoding gate). RY+RZ per qubit per layer gives exactly 2 trainable parameters per qubit per layer, a clean, documented parameter-count formula (p=2nL), and is a standard, simple hardware-efficient-ansatz choice; a full U3 gate (3 parameters) was not used because it would break that clean formula and wasn't necessary for the comparison being made.

Q: Why CZ and not CNOT for entanglement?

A: CZ is symmetric in its two qubits (order doesn't matter) and has no trainable parameter, making the ring topology trivial to define without a control/target asymmetry choice; it entangles the qubits exactly the same way CNOT does up to local basis changes (both are maximally entangling 2-qubit Clifford gates) -- the project's conclusions would not change qualitatively with CNOT instead.

Q: Why is the entangling gate's edge ring different for 2 qubits?

A: A ring over 2 qubits would naively list edges (0,1) and (1,0) -- but that is the same physical edge applied twice (CZ is symmetric), which would silently double the entangling operation. The code explicitly checks for n_qubits==2 and applies that single edge only once; this is directly unit-tested.

Q: What exactly does 'barren plateau' mean, formally?

A: A region of parameter space where, for almost all randomly chosen parameters, the gradient of the cost function with respect to almost every parameter has a variance that shrinks exponentially in qubit count (and/or depth), making the gradient statistically indistinguishable from sampling/numerical noise.

Q: Why does a barren plateau happen mathematically?

A: For circuits that approximate Haar-random unitaries (form an approximate 2-design), integrating the gradient over the Haar measure shows its variance scales as roughly 1/2^n -- exponentially small in qubit count n. The mean gradient is zero by symmetry, so a shrinking variance concentrates almost all probability mass arbitrarily close to zero gradient.

Q: Does your project prove barren plateaus mathematically, or just observe them?

A: Observes them empirically, via the dedicated gradient-variance diagnostic (Section 3.5/15.7) -- this project is an experimental/empirical study, not a new theoretical proof; it cites and relies on McClean et al. (2018) and related work for the theory, and generates its own fresh measurements with this exact architecture to confirm the effect is really present here.

Q: Why measure only qubit 0 (a 'local' observable) instead of all qubits?

A: Cerezo et al. (2021) showed global cost functions (observables acting on all qubits) cause barren plateaus even in shallow circuits, while local cost functions delay the onset to larger depths -- using a single-qubit readout was a deliberate design choice to avoid stacking that extra plateau-inducing effect on top of the depth/width effect actually being studied.

Q: What is the difference between Direct Training and Layer-wise Training in one sentence each?

A: Direct Training builds the full L-layer circuit and trains every parameter together from one random initialization. Layer-wise Training starts at 1 layer, trains it, freezes it, adds a new near-zero layer, trains only that new layer, and repeats until reaching L layers.

Q: How is 'freezing' actually enforced -- couldn't the optimizer still nudge old parameters a little?

A: No -- the frozen layers are passed into the circuit function as a plain constant NumPy array, and the automatic-differentiation call (qml.grad) is only told to differentiate with respect to the new layer's array. The old parameters never appear in the computed gradient at all, so Adam's update (which only acts on the gradient it receives) cannot move them. This is verified by an exact equality check after every stage in every real run, and by an automated test.

Q: Is 'near-zero' initialization for new layers the same as 'identity' initialization?

A: No, and this is a documented caveat, not an oversight: the new layer's CZ gates are applied regardless of how close the rotation angles are to zero, so the new layer is never exactly an identity operation. The project measured a small but real loss shift immediately after every layer insertion to confirm this.

Q: Why MSE loss and not binary cross-entropy?

A: The circuit's raw output (a Pauli-Z expectation value) is naturally in [-1,+1], which matches directly onto a ±1 label with simple squared-error, without needing an extra sigmoid/log-loss layer that would require a probability interpretation the raw expectation value doesn't actually have.

Q: What's the difference between backprop and the parameter-shift rule, and why do you need both?

A: Backprop differentiates straight through the simulator's internal computation -- exact and fast, but only possible because this is a classical simulation; a real device can't do it. Parameter-shift gets the exact gradient of a gate by evaluating the whole circuit twice more at the parameter shifted by ±π/2 and taking a half-difference -- more expensive (2 extra circuit runs per parameter) but implementable on real hardware and valid whenever the circuit must be measured with finite shots or includes noise, where backprop-through-the-simulator would be unrealistic.

Q: Why is the shots/noise experiment so much slower than the main grid?

A: It must use the parameter-shift rule (not backprop), whose cost scales with roughly 2×(number of trainable parameters) extra circuit evaluations per sample per update -- for a 48-parameter circuit, about 97 evaluations per sample versus effectively 1 for backprop. Measured directly: one depth-2 (16-parameter) noisy run took 124 seconds; this is why depth 6 was dropped from the quick-mode noise subset.

Q: What does PCA do here and why is it needed?

A: PCA (Principal Component Analysis) linearly projects the 61-dimensional processed feature vector onto the n_qubits directions of largest variance in the training data, because the circuit needs exactly one input feature per qubit (one RY encoding gate per qubit), and there are always far more raw/processed features than qubits.

Q: How much information does PCA throw away?

A: A lot, and this is stated explicitly: 4-qubit PCA keeps only 41.4% of the processed feature variance; 6-qubit PCA keeps 53.8%. This is the single biggest explanation for why even the best VQC result (around 0.59-0.60 balanced accuracy) falls well short of full-feature classical Logistic Regression (0.690).

Q: Why scale angles to [0, pi] and not [0, 2*pi] or [-pi, pi]?

A: RY(θ) sweeps a qubit continuously from |0⟩ at θ=0 to |1⟩ at θ=π; using the full [0,π] range uses exactly one full, non-redundant sweep of that rotation for the encoding, without wrapping past a point that would alias back toward states already covered.

Q: How do you prevent data leakage, precisely?

A: Every fitted transform (one-hot encoder, standard scaler, PCA, angle min-max scaler) is fit using only the training split; validation and test data are only ever passed through '.transform()', never '.fit()'. This is enforced by the code structure and directly checked by an automated test that proves PCA fit on different training subsets gives different components (i.e. it genuinely only learns from what it's given).

Q: Why a 70/15/15 split, and why stratified?

A: 70/15/15 is a standard, defensible default for a dataset this size, giving reasonably sized validation and test sets (150 each) while keeping most data for training. 'Stratified' means the 70/30 good/bad class ratio is preserved in every split, so small-sample class-imbalance artifacts don't distort the validation/test sets differently from training.

Q: What is the UCI cost matrix and why use it?

A: The dataset's own official documentation states that misclassifying a bad-credit applicant as good costs 5x more than the reverse mistake (losing loan principal vs. losing a potential customer) -- using this real, documented asymmetry (rather than treating both error types equally) gives a cost-sensitive metric closer to real-world financial impact.

Q: Your Direct Training got HIGHER raw accuracy than Layer-wise. Doesn't that mean Direct Training won?

A: No -- with a 700/300 class split, a model that always predicts the majority class scores ~70% raw accuracy while discriminating nothing. Direct Training's validation balanced accuracy stayed near chance (~0.51) throughout training, which is the signature of exactly that majority-class collapse. Layer-wise's lower raw accuracy came with meaningfully higher balanced accuracy and F1, meaning it was actually distinguishing some bad-credit cases. We treat balanced accuracy/F1, which correct for class imbalance, as the metrics that matter here, not raw accuracy.

Q: Layer-wise had a HIGHER plateau fraction than Direct. Doesn't that contradict your whole hypothesis?

A: It shows the hypothesis is nuanced, not that it's wrong. Layer-wise deliberately restarts near-zero at the start of every new stage, and that near-zero region genuinely does have small gradients while the new layer 'warms up' -- we measured this directly as small loss jumps at every layer insertion. So Layer-wise trades a larger share of small-gradient individual updates for a better-conditioned overall landscape at the start of each stage. Both are real, measured effects; we report both rather than picking the one that makes the story cleaner.

Q: Are your results statistically significant?

A: No, not at the quick-mode seed count. All 18 paired statistical tests (6 depth/qubit cells × 3 metrics) gave $p \geq 0.05$ (mostly ≥ 0.495), using a two-sided permutation test predeclared before the experiment ran. The point estimates consistently favour Layer-wise on balanced accuracy and F1, which is suggestive given how consistent the direction is, but with only 3 paired seeds we explicitly do not claim statistical significance we did not measure.

Q: Why only 3 seeds instead of the 10 you say 'full mode' would use?

A: A measured pilot (not a guess) showed the full grid (10 seeds, depths to 24, all 3 qubit counts) would take roughly 3.5-4 hours of CPU time for the noiseless grid alone, before the shots/noise sub-experiments. Given the time available for this delivery, quick mode (3 seeds, depths up to 12, completing in under 6 minutes end-to-end) was used to produce real, honestly-reported results, while the full grid was left fully implemented, configured, and resumable for a future run.

Q: Did you run this on real quantum hardware?

A: No. Everything runs on PennyLane's classical simulators -- default.qubit for exact and finite-shot simulation, default.mixed (a density-matrix simulator) for the depolarizing-noise experiment. This is stated explicitly as a limitation; the noise experiment is a simulated noise model, not a real-hardware result.

Q: What is 'default.mixed' and why is it needed only for the noise experiment?

A: It's a density-matrix simulator (tracks a mixed quantum state, a probabilistic ensemble of pure states) rather than a single statevector. It's required specifically because simulating a noise channel (like depolarizing noise) means the final state is generally mixed, not pure, which a plain statevector simulator (default.qubit) cannot represent.

Q: What does the depolarizing noise channel actually do?

A: With probability p, it replaces a qubit's state with the maximally mixed state (equivalent to applying a random Pauli X, Y, or Z error with some probability); with probability 1-p it leaves the qubit untouched. In this project it's applied after every gate on every qubit, at probabilities 0, 0.001, and 0.01.

Q: Why did depolarizing noise show basically no effect in your results?

A: The tested configuration was shallow (depth 2) and the tested probabilities were low (≤0.01) -- noise effects accumulate with circuit depth (more gates = more chances for a noise event), and Wang et al. (2021) show gradient/output degradation from noise requires depth to scale up before it dominates. At this shallow depth and low noise level, the continuous loss shifted only slightly (~1e-4 to 1e-3) -- real but too small to flip any individual discrete prediction. We verified this against the raw per-seed data to confirm it wasn't a bug before reporting it.

Q: What classical baselines did you use and why those two?

A: Logistic Regression (a simple, strong, standard linear baseline) and a small one-hidden-layer MLP (a minimal nonlinear classical neural network), each evaluated on both the full 61-feature representation and the same PCA-reduced representation the VQC sees, so the comparison is explicit about which input each model received.

Q: Your MLP baseline underperformed even Logistic Regression. Isn't that a weak baseline?

A: Yes, and we say so explicitly rather than hiding or re-tuning it to look better -- it's plausibly explained by early stopping on a small validation set, and it's reported as-is. A weaker-than-ideal MLP doesn't change the main VQC-vs-VQC comparison, which is the project's actual research question; it's provided for context, not as the primary result.

Q: What would you do differently with more time (full mode)?

A: Run the full 10-seed, depth-up-to-24, all-3-qubit-count grid for real statistical power; extend the shots/noise sub-experiments to the full predeclared subset; and potentially test the optional 'final blockwise fine-tuning' extension to layer-wise training mentioned in the methodology but deliberately not used in the primary comparison.

Q: How do you know your code is actually correct, not just that it ran without crashing?

A: A 45-test automated pytest suite checks: correct dataset schema and label mapping, no train/val/test index overlap, no preprocessing leakage, correct input dimension per qubit count, the exact trainable-parameter count formula, circuit output bounded in [-1,1], finite loss/gradients, exact bit-for-bit reproducibility under a fixed seed, exact frozen-parameter enforcement in layer-wise training, correct metric values on hand-computed toy examples, correct table/figure regeneration from saved CSVs, and graceful handling of a simulated run failure. All 45 currently pass.

Q: Did you find and fix any real bugs while building this?

A: Yes, and we can name them precisely: (1) an Excel export crash caused by a multi-index column table, (2) a pandas groupby silently dropping the 'analytic' (no-shots) rows from a results table because they were stored as NaN, (3) a wall-clock/success-rate plot that averaged only the first qubit count's results instead of truly averaging across qubit counts, and (4) a measurement bug where the 'loss immediately after layer insertion' was accidentally computed using only the old frozen layers, making it identical to the previous stage's final loss and unable to show the architectural caveat it was meant to demonstrate. All four were caught during our own result review, fixed, and locked in with new regression tests.

Q: What is the single most important figure you would show first?

A: figures/val_accuracy_vs_updates_nq4_d12.png, because it visually demonstrates the accuracy-vs-balanced-accuracy paradox (Section 15.4) that reframes the entire 'who won' question, and figures/gradient_variance_diagnostic.png, because it is the project's own direct, dataset-independent proof that a barren-plateau effect genuinely exists in this architecture.

Q: In one sentence, what is your overall conclusion?

A: Layer-wise Training showed a consistent, speed-and-balanced-accuracy advantage over Direct Training that is well-supported by this project's own gradient-variance evidence of a barren plateau, but the advantage is not yet statistically significant at the seed count actually tested, and neither method solved the underlying classification task within the compute budget used.

Q: Walk me through exactly what happens, in order, when you run 'python run_project.py --mode quick'.

A: (1) src/data_retrieval.py fetches/validates the real dataset, caching it to data/raw/. (2) src/preprocessing.py builds a leakage-safe pipeline (one-hot + scale + PCA + angle-scale, fit on the training split only) for each qubit count. (3) src/baselines.py trains Logistic Regression and an MLP. (4) src/experiment_runner.py runs the full Direct-vs-Layer-wise grid via src/training.py and src/circuits.py, logging every run to results/quick/main_grid_results.csv and its full history to results/quick/histories/*.json, skipping anything already marked 'success'. (5)-(7) the same runner runs the shots experiment, the noise experiment, and src/gradient_diagnostics.py. (8) src/circuits.py draws a real circuit diagram per configuration. (9) src/export.py builds the 9 CSV tables and the Excel workbook from those saved CSVs, src/plotting.py builds every figure from the same saved CSVs and histories, and a final result_manifest.json and run_summary.json are written.

Q: You added confusion-matrix figures after your first pass. How did you find that gap, and how did you fix it without retraining anything?

A: src/plotting.py had always had a plot_confusion_matrix() function, but nothing in run_project.py ever called it -- found by checking the actual figures/ folder against the project's own mandatory-graphs checklist. The fix (generate_confusion_matrices() in run_project.py) does not retrain anything: for each protocol it finds the best saved run by test_balanced_accuracy in the already-completed results CSV, reloads that exact run's final_weights from its saved JSON history, rebuilds the same circuit with src.circuits.make_qnode, and re-evaluates it once on the untouched test set to get the confusion matrix -- and separately re-runs the (fast, deterministic) classical baselines once more for their confusion matrices. This is also a good example of how the project's save-everything/resumable design (every run's final weights saved to JSON) made a late addition like this cheap and safe to add.

24. Glossary

Term	Definition
Ansatz	The specific, fixed structure of trainable gates used in a VQC (here: RY+RZ per qubit per layer, CZ ring).
Barren plateau	A region of parameter space where gradients vanish (exponentially, in the worst case) making optimization stall.
Circuit depth	The number of trainable layers in the circuit (this project tests depths 2, 6, 12; up to 24 in the full grid and the gradient diagnostic).
Entanglement	A quantum correlation between qubits that cannot be described by treating them as independent systems; produced here by CZ gates.
Expectation value	The probability-weighted average measurement outcome of an observable; here, of Pauli-Z on one qubit, always in [-1,+1].
Parameter-shift rule	An exact method to compute a gate's gradient by evaluating the circuit at the parameter shifted by +pi/2 and -pi/2.
PCA (Principal Component Analysis)	A linear dimensionality-reduction technique that projects data onto its directions of largest variance.
Qubit	A two-level quantum system, the quantum analogue of a classical bit.
Shots	The number of times a quantum circuit is actually measured/sampled to estimate an expectation value on real or simulated finite-sampling hardware.
Stratified split	A train/val/test split that preserves the original class proportions in every subset.
Variational Quantum Circuit (VQC)	A quantum circuit with trainable classical parameters, optimized via a classical loss function and optimizer.